

Library functions

The C language is accompanied by a number of standard *library functions* which carry out various useful tasks. In particular, all input and output operations (*e.g.*, writing to the terminal) and all math operations (*e.g.*, evaluation of sines and cosines) are implemented by library functions.

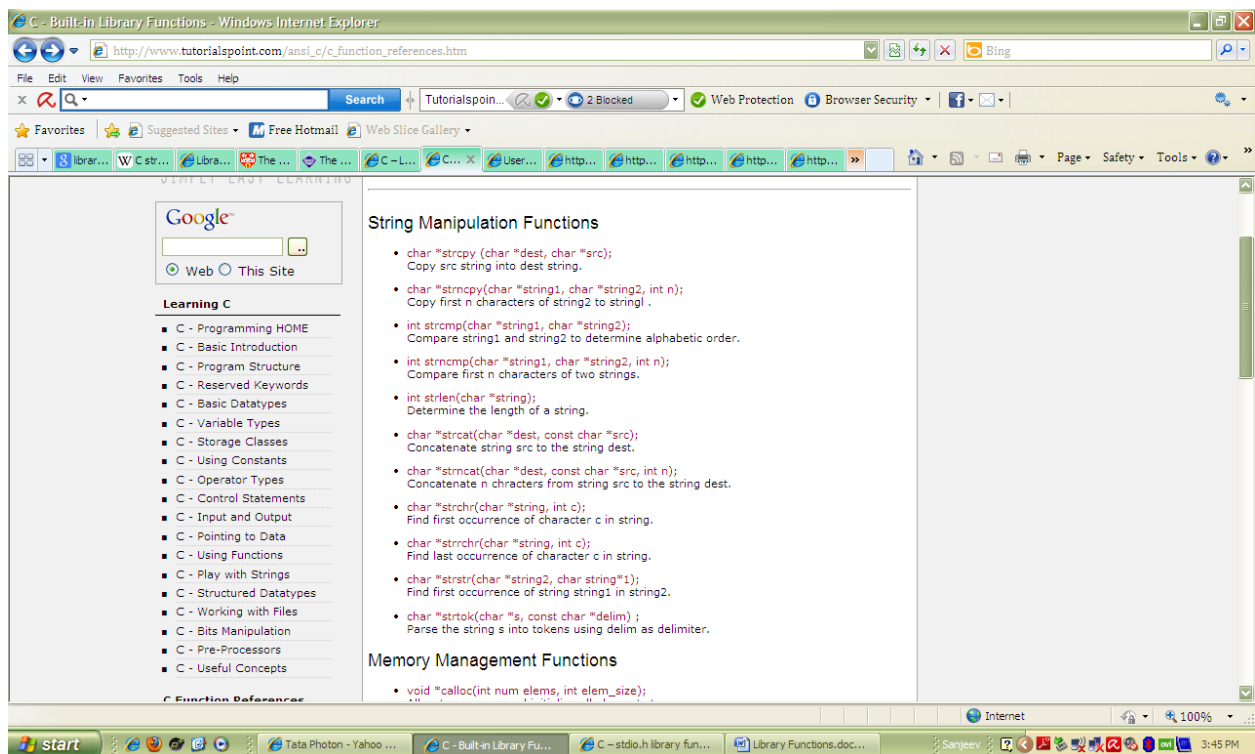
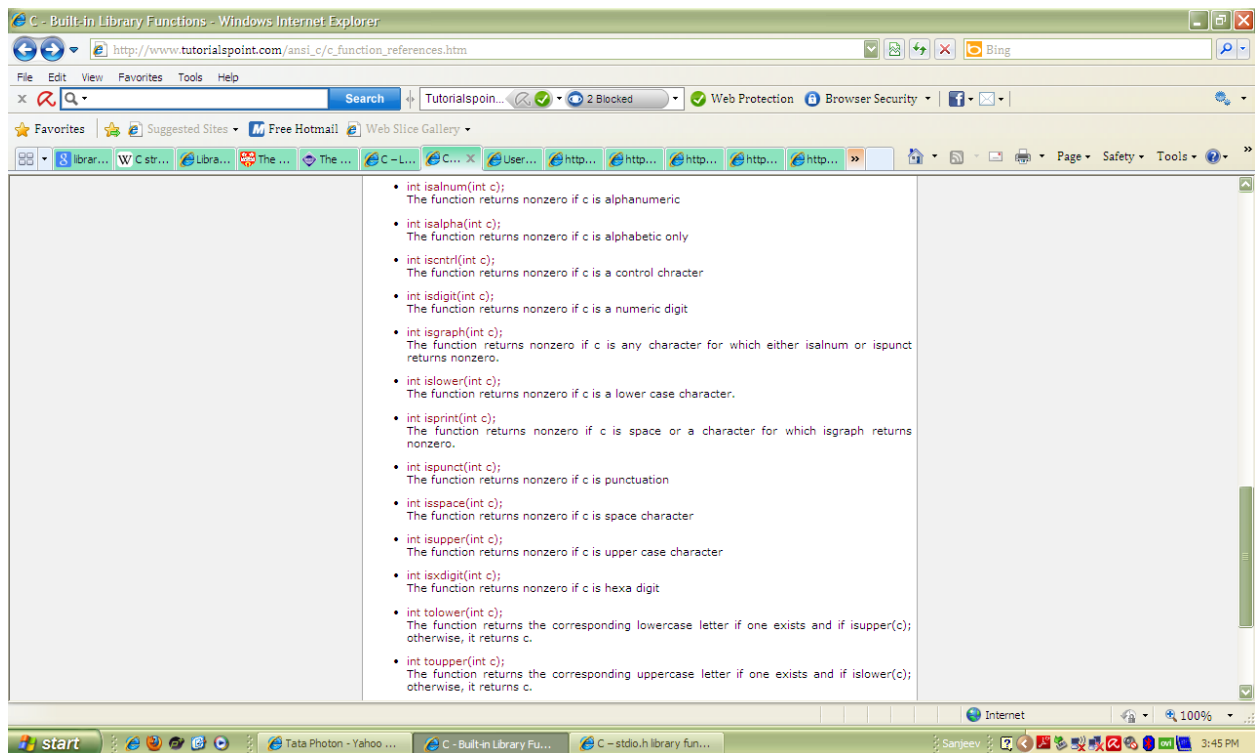
In order to use a library function, it is necessary to call the appropriate *header file* at the beginning of the program. The header file informs the program of the name, type, and number and type of arguments, of all of the functions contained in the library in question. A header file is called via the preprocessor statement

```
#include <filename>
```

where `filename` represents the name of the header file.

A library function is accessed by simply writing the function name, followed by a list of *arguments*, which represent the information being passed to the function. The arguments must be enclosed in parentheses, and separated by commas: they can be constants, variables, or more complex expressions. Note that the parentheses must be present even when there are no arguments.

Character Functions



Stdio.h: This is standard input/output header file in which Input/Output functions are declared and defined

S.no	Function	Description
1	printf()	This function is used to print the character, string, float, integer, octal and hexadecimal values onto the output screen
2	scanf()	This function is used to read a character, string, numeric data from keyboard.
3	getc()	It reads character from file
4	gets()	It reads line from keyboard
5	getchar()	It reads character from keyboard
6	puts()	It writes line to o/p screen
7	putchar()	It writes a character to screen
8	clearerr()	This function clears the error indicators
9	f open()	All file handling functions are defined in stdio.h header file
10	f close()	closes an opened file
11	getw()	reads an integer from file
12	putw()	writes an integer to file
13	f getc()	reads a character from file
14	putc()	writes a character to file
15	f putc()	writes a character to file
16	f gets()	reads string from a file, one line at a time
17	f puts()	writes string to a file

18	f eof()	finds end of file
19	f getchar	reads a character from keyboard
20	f getc()	reads a character from file
21	f printf()	writes formatted data to a file
22	f scanf()	reads formatted data from a file
23	f getchar	reads a character from keyboard
24	f putchar	writes a character from keyboard
25	f seek()	moves file pointer position to given location
26	SEEK_SET	moves file pointer position to the beginning of the file
27	SEEK_CUR	moves file pointer position to given location
28	SEEK_END	moves file pointer position to the end of file.
29	f tell()	gives current position of file pointer
30	rewind()	moves file pointer position to the beginning of the file
31	putc()	writes a character to file
32	sprint()	writes formatted output to string
33	sscanf()	Reads formatted input from a string
34	remove()	deletes a file
35	fflush()	flushes a file

Stdlib.h: This header file contains general functions used in C programs

S.no	Function	Description
1	malloc()	This function is used to allocate space in memory during the execution of the program.
2	calloc()	This function is also like malloc () function. But calloc () initializes the allocated

		memory to zero. But, malloc() doesn't
3	realloc()	This function modifies the allocated memory size by malloc () and calloc () functions to new size
4	free()	This function frees the allocated memory by malloc (), calloc (), realloc () functions and returns the memory to the system.
5	abs()	This function returns the absolute value of an integer . The absolute value of a number is always positive. Only integer values are supported in C.
6	div()	This function performs division operation
7	abort()	It terminates the C program
8	exit()	This function terminates the program and does not return any value
9	system()	This function is used to execute commands outside the C program.
10	atoi()	Converts string to int
11	atol()	Converts string to long
12	atof()	Converts string to float
13	strtod()	Converts string to double
14	strtol()	Converts string to long
15	getenv()	This function gets the current value of the environment variable
16	setenv()	This function sets the value for environment variable
17	putenv()	This function modifies the value for environment variable
18	perror()	This function displays most recent error that happened during library function call.
19	rand()	This function returns the random integer numbers
20	delay()	This function Suspends the execution of the program for particular time

Conio.h: This is console input/output header file

S.no	Function	Description
1	clrscr()	This function is used to clear the output screen.
2	getch()	It reads character from keyboard
3	getche()	It reads character from keyboard and echoes to o/p screen
4	textcolor()	This function is used to change the text color
5	textbackground()	This function is used to change text background

Ctype.h: All character handling functions are defined in this header file

S.no	Function	Description
1	<u>isalpha()</u>	checks whether character is alphabetic
2	<u>isdigit()</u>	checks whether character is digit
3	<u>isalnum()</u>	checks whether character is alphanumeric
4	<u>isspace()</u>	checks whether character is space
5	<u>islower()</u>	checks whether character is lower case
6	<u>isupper()</u>	checks whether character is upper case
7	<u>isxdigit()</u>	checks whether character is hexadecimal
8	<u>iscntrl()</u>	checks whether character is a control character
9	<u>isprint()</u>	checks whether character is a printable character
10	<u>ispunct()</u>	checks whether character is a punctuation
11	<u>isgraph()</u>	checks whether character is a graphical character
12	<u>tolower()</u>	checks whether character is alphabetic & converts to lower case
13	<u>toupper()</u>	checks whether character is alphabetic & converts to upper case

Math.h : All maths related functions are defined in this header file

S.no	Function	Description
1	<u>floor ()</u>	This function returns the nearest integer which is less than or equal to the argument passed to this function.
2	<u>round ()</u>	This function returns the nearest integer value of the float/double/long double argument passed to this function. If decimal value is from ".1 to .5", it returns integer value less than the argument. If decimal value is from ".6 to .9", it returns the integer value greater than the argument.
3	<u>ceil ()</u>	This function returns nearest integer value which is greater than or equal to the argument passed to this function.
4	<u>sin ()</u>	This function is used to calculate sine value.
5	<u>cos ()</u>	This function is used to calculate cosine.
6	<u>cosh ()</u>	This function is used to calculate hyperbolic cosine.
7	<u>exp ()</u>	This function is used to calculate the exponential "e" to the x th power.

8	tan ()	This function is used to calculate tangent.
9	tanh ()	This function is used to calculate hyperbolic tangent.
10	sinh ()	This function is used to calculate hyperbolic sine.
11	log ()	This function is used to calculates natural logarithm.
12	log10 ()	This function is used to calculates base 10 logarithm.
13	sqrt ()	This function is used to find square root of the argument passed to this function.
14	pow ()	This is used to find the power of the given number.
15	trunc ()	This function truncates the decimal value from floating point value and returns integer value.

String.h: All string related functions are defined in this header file.

S.no	string functions	Description
1	strcat(str1, str2)	Concatenates str2 at the end of str1.
2	strcpy(str1, str2)	Copies str2 into str1
3	strlen(str1)	gives the length of str1.
4	strcmp(str1, str2)	Returns 0 if str1 is same as str2. Returns <0 if str1 < str2. Returns >0 if str1 > str2.
5	strchr(str1,char)	Returns pointer to first occurrence of char in str1.
6	strstr(str1, str2)	Returns pointer to first occurrence of str2 in str1.
7	strcmpi(str1,str2)	Same as strcmp() function. But, this function negotiates case. “A” and “a” are treated as same.
8	strdup()	duplicates the string
9	strlwr()	converts string to lowercase
10	strncat()	appends a portion of string to another
11	strncpy()	copies given number of characters of one string to another
12	strrchr()	last occurrence of given character in a string is found
13	strrev()	reverses the given string
14	strset()	sets all character in a string to given character
15	strupr()	converts string to uppercase
16	strtok()	tokenizing given string using delimiter
17	memset()	It is used to initialize a specified number of bytes to null or any other

		value in the buffer
18	memcpy()	It is used to copy a specified number of bytes from one memory to another
19	memmove()	It is used to copy a specified number of bytes from one memory to another or to overlap on same memory.
20	memcmp()	It is used to compare specified number of characters from two buffers
21	memicmp()	It is used to compare specified number of characters from two buffers regardless of the case of the characters
22	memchr()	It is used to locate the first occurrence of the character in the specified string

Process.h:

1	Void c_exit(void)	Performs _exit cleanup without terminating the program. _c_exit performs the same cleanup as _exit, except that it does not terminate the calling process.
2	void _cexit(void);	Performs exit cleanup without terminating the program. _cexit performs the same cleanup as exit, closing all files but without terminating the calling process. The _cexit function calls any registered "exit functions" (posted with atexit). Before _cexit returns, it flushes all input/output buffers and closes all streams that were open.
3	void _endthread(void);	Terminates execution of a thread. The _endthread function terminates the currently executing thread by closing the thread handle and calling the ExitThread API. The thread must have been started by an earlier call to _beginthread or _beginthreadNT.. _endthread is called automatically by the runtime library when your thread function terminates. This function is available in the multithread libraries; it is not in the single-threaded libraries.
4	void _endthreadex(unsigned thread_retval);	Terminates execution of the current thread by calling the ExitThread API, but without closing the handle.
5	void *_expand(void *block, size_t size);	Description Grows or shrinks a heap block in place.

		This function attempts to change the size of an allocated memory block without moving the block's location in the heap. The data in the block are not changed, up to the smaller of the old and new sizes of the block. The block must have been allocated earlier with malloc, calloc, or realloc, and must not have been freed.
6	int cwait(int *statloc, int pid, int action);	Waits for child process to terminate.

Token pasting operator

ANSI has introduced a well-defined token-pasting operator, ##, which can be used like this:

```
#define f(g,g2) g##g2

main()

{

    int var12=100;

    printf("%d",f(var,12));

}
```

O/P

100